# Inference in High Dimensional Data using Python

Narayan Sarkar, IBM

Master of Statistics, Indian Statistical Institute

sarkar.narayan@gmail.com

March 22, 2026

## 1 High Dimensional Data Analysis with Python

### 1.1 Regression

We will use baseball league data referred as Hitters data set in (**?**). We will predict the salary of baseball players based on players' career and performance data in previous year. The data set is available in the ISLP Python package (**?**). Technically, this is not an example of high dimensional data, but it is a good data set with adequate number of features to apply the high dimensional regression techniques which can produce better results than the classical least square regression technique.

We will implement many of the techniques discussed in **??** using (**?**) Python libraries.

Let us import the required python libraries.

```
import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
import sklearn.model_selection as skm
from ISLP import load_data
import sklearn.linear_model as skl
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from ISLP.models import ModelSpec as MS
from sklearn.pipeline import Pipeline
```

We read Hitters data and exclude the rows having missing values for one or more variables.

```
Hitters = load_data('Hitters')
Hitters = Hitters.dropna()
Hitters.shape
```

```
(263, 20)
```

#### 1.1.1 Ridge Regression

We will use sklearn.linear_model package to fit the ridge regression. Let us name this skl as abbreviation for this package. We use the learning lab example with Hitters data from **?**. We first construct the model matrix $X$ excluding the intercept and then standardize the columns of $X$.

```
x_cols = Hitters.columns.drop('Salary')
design = MS(x_cols).fit(Hitters)
Y = np.array(Hitters['Salary'])
D = design.transform(Hitters)
X = D.drop('intercept', axis =1)
scaler = StandardScaler()
Xs = scaler.fit_transform(X)
```

We will use the functions and parameters as listed below:

1. skl.ElasticNet() - Python package to Fit ridge and the lasso models
2. skl.ElasticNet.path() - To fit paths for ridge, lasso and hybrid mixture (Elastic Net) models
3. *l1_ratio* - Parameter to set the value for ridge, lasso or Elastic Net model. *l1_ratio* $= 0$ is for ridge regression.

We implement the function skl.ElasticNet.path() for a grid of values of $\lambda = 10^8$ to $\lambda = 10^{-2}$ scaled by the standard deviation of $y$. This covers full range of models starting the null model with only intercept.

```
lambdas = 10 ** np.linspace(8, -2, 100) / Y.std()
soln_array = skl.ElasticNet.path(Xs, Y, l1_ratio=0., alphas=lambdas)[1]
soln_array.shape
```

```
(19, 100)
```

soln_array above formed a $19 \times 100$ matrix, each column representing a vector of ridge regression coefficients for each value of $\lambda$ in the sequence of 100 $\lambda$ values and each row representing a predictor of 19 predictors. We transpose this matrix and convert to a data frame.

```
soln_path = pd.DataFrame(soln_array.T,
                         columns=X.columns,
                         index=-np.log(lambdas))
soln_path.index.name = 'negative log(lambda)'
soln_path
```

We plot the paths in the figure 1 to check how the coefficients are varing with $\lambda$. The coefficient estimates in terms of $L_2$ norm are shrinking when the value of $\lambda$ is increased.

For a sequence of $\lambda$ values, we can fit ridge, lasso, or elastic net model and create a solution or regularization path for regression coefficients along the sequence of $\lambda$ values. ElasticNetCV function implements a specialized technique to fit a regularization path and use cross-validation to choose a suitable value of $\lambda$.

We have used Pipeline() object in sklearn to implement feature normalization and fit the ridge regression model. We set up the kfold object for 5-fold cross-validation.

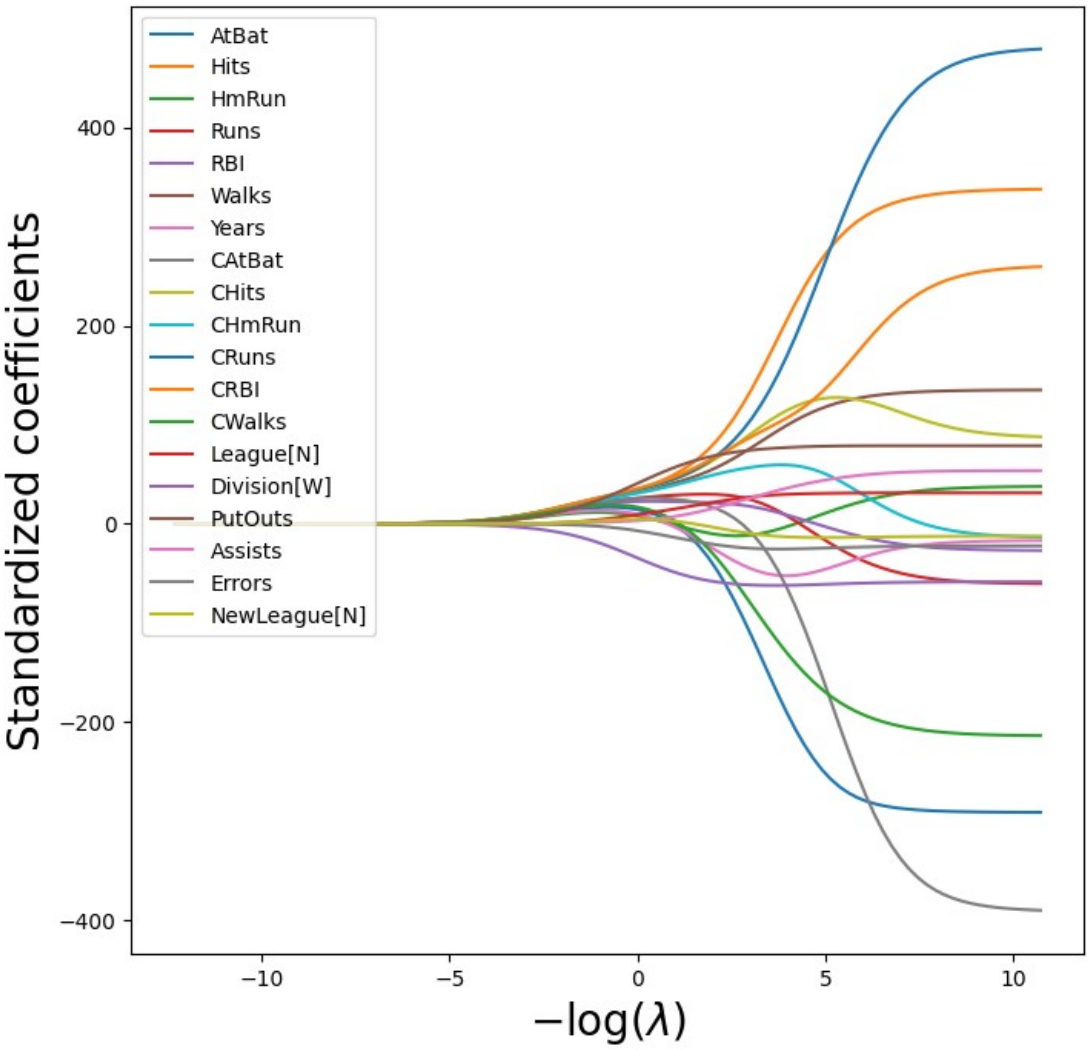


Figure 1: The standardized ridge regression coefficients are displayed as a function of $\lambda$ in the log scale

```
K = 5
kfold = skm.KFold(K,random_state=0,shuffle=True)
ridgeCV = skl.ElasticNetCV(alphas=lambdas,
                           l1_ratio=0,
                           cv=kfold)
pipeCV  = Pipeline(steps = [('scaler', scaler), ('ridge', ridgeCV)])
pipeCV.fit(X,Y)
```

The figure 2 shows the plot for cross-validation errors to see the value of $\lambda$ that results in the smallest cross-validation error.

The below source code provides the value of $\lambda$ that results the smallest cross-validation error and the value of smallest cross-validation error with this value of $\lambda$.

```
tuned_ridge.alpha_
0.01185247763144249

np.min(tuned_ridge.mse_path_.mean(1))
115526.70630987761
```

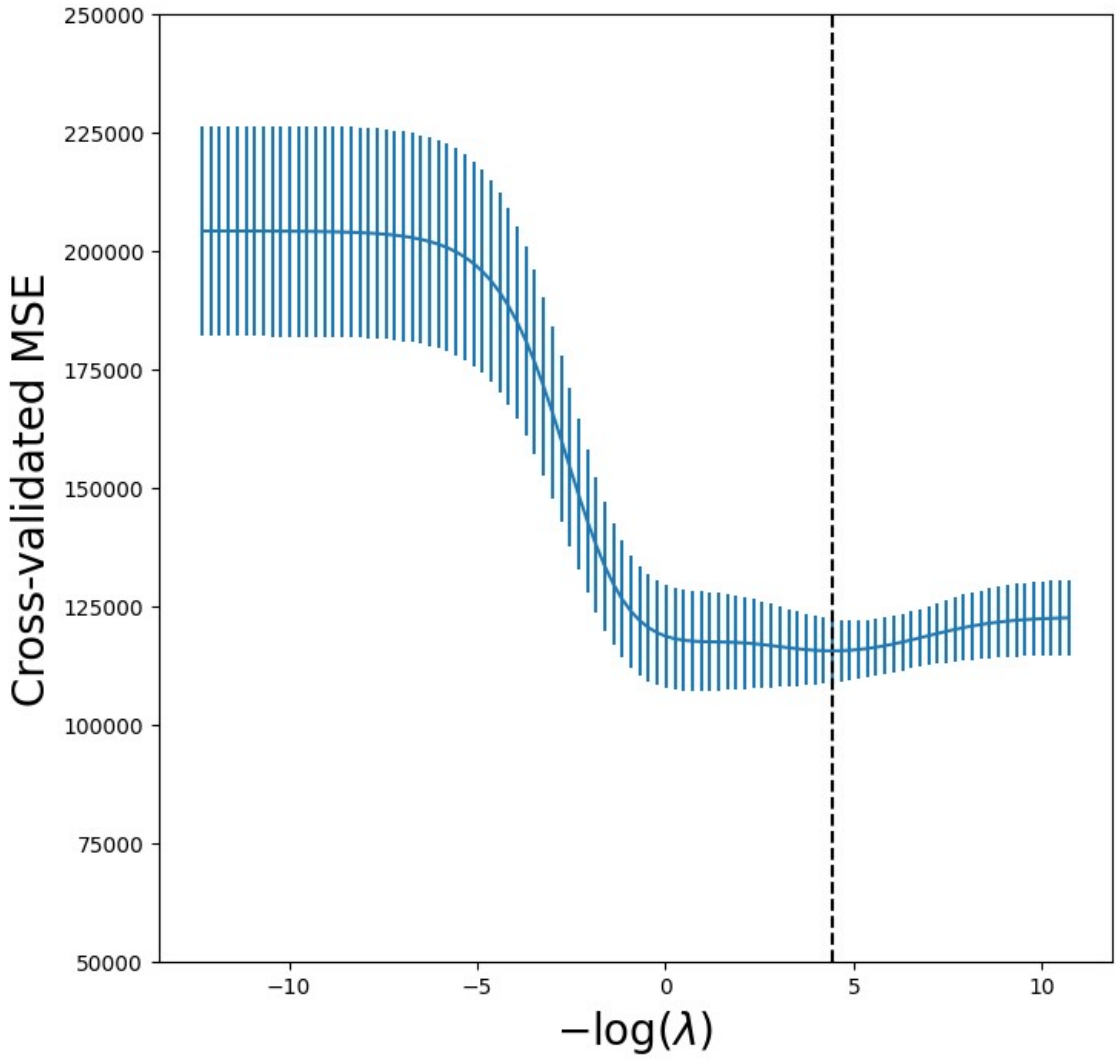


Figure 2: Cross-validation error against $\lambda$ in the log scale for ridge regression fit

For setting, $\lambda = 1.19e - 02$, we can see the coefficient estimates on the entire data in tuned_ridge.coef_. Since ridge regression does not perform variable selection, none of the coefficients is zero.
We split the data with 75% for training and 25% for test. Then we perform a 5-fold cross-validation on the training data to tune for ridge regression model, and evaluate its performance on the test data.

```
outer_valid = skm.ShuffleSplit(n_splits=1, test_size=0.25, random_state=1)
inner_cv = skm.KFold(n_splits=K, shuffle = True, random_state=2)
ridgeCV = skl.ElasticNetCV(alphas=lambdas, l1_ratio=0, cv=inner_cv)
pipeCV = Pipeline(steps=[('scaler', scaler), ('ridge', ridgeCV)])
```

The test MSE is 132393.84.

```
results = skm.cross_validate(pipeCV,
                             X,
                             Y,
                             cv = outer_valid,
                             scoring = 'neg_mean_squared_error')
-results['test_score']

array([132393.84003227])
```

### 1.1.2 The Lasso

We will use the lasso on the same Hitters data to check if we can have the model with better performance and more interpretable than ridge regression. Let us use the function the ElasticNetCV() with the argument $l1_ratio = 1$ to fit a lasso model.

```
lassoCV = skl.ElasticNetCV(n_alphas=100, l1_ratio=1, cv=kfold)
pipeCV = Pipeline(steps=[('scaler', scaler), ('lasso', lassoCV)])
pipeCV.fit(X,Y)
tuned_lasso = pipeCV.named_steps['lasso']
tuned_lasso.alpha_

3.1472370031649866
```

For each value of $\lambda$, let us get access to the vector of lasso regression coefficients from the matrix, soln_array.

```
lambdas, soln_array = skl.Lasso.path(Xs,
                                     Y,
                                     l1_ratio=1,
                                     n_alphas=100)[:2]

soln_path = pd.DataFrame(soln_array.T,
                         columns=X.columns,
                         index=-np.log(lambdas))
```

We can see the plots of standardized coefficients in figure 3. As the tuning $\lambda$ varied, some of the coefficients are exactly zero.
We plot the cross-validation error in figure 4 and select the tuning parameter, $\lambda$ for which cross-validation error is minimum.
The smallest cross-validated error is 114690.7312 at $\lambda = 3.1472$

```
np.min(tuned_lasso.mse_path_.mean(1))
114690.73118253658
```

We can see that the lasso has an advantage over ridge regression for selecting variables in the model. The lasso has made 6 out of 19 coefficient estimates to zero. Corresponding the value of $\lambda$ chosen using cross-validation, only 13 variables are selected.

```
sum(tuned_lasso.coef_ == 0)
6
```

We now evaluate the test error for the lasso model with the value of $\lambda$ chosen by cross-validation. We split training and test sets and run cross-validation on the training set.The test MSE is 133473.14.

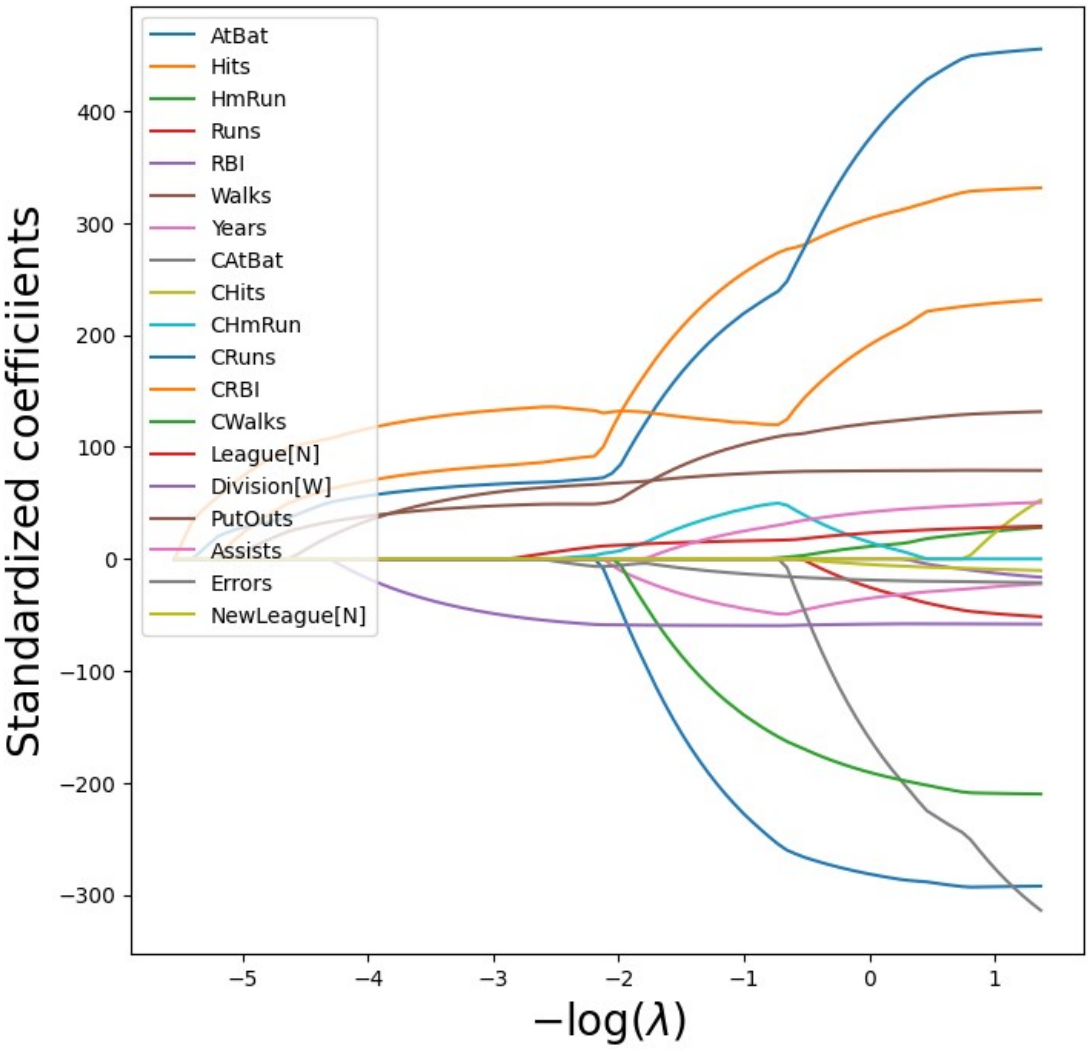


Figure 3: The standardized coefficients of the lasso fit on the Hitters data set against the sequence of values of $\lambda$

```
outer_valid = skm.ShuffleSplit(n_splits=1, test_size=0.25, random_state=1)
inner_cv = skm.KFold(n_splits=K, shuffle = True, random_state=2)
lassoCV = skl.ElasticNetCV(n_alphas=100, l1_ratio=1, cv=inner_cv)
pipeCV = Pipeline(steps=[('scaler', scaler), ('lasso', lassoCV)])
results = skm.cross_validate(pipeCV,
                             X,
                             Y,
                             cv = outer_valid,
                             scoring = 'neg_mean_squared_error')
-results['test_score']

array([133473.13684813])
```

### 1.1.3 Principal Components Regression (PCR)

We use PCA() from the sklearn.decomposition module to perform Principal components regression (PCR). We use the data Hitters to predict Salary applying the PCR. We fit the regression model using LinearRegression() with principal components. We also standardize the data to perform the PCA.

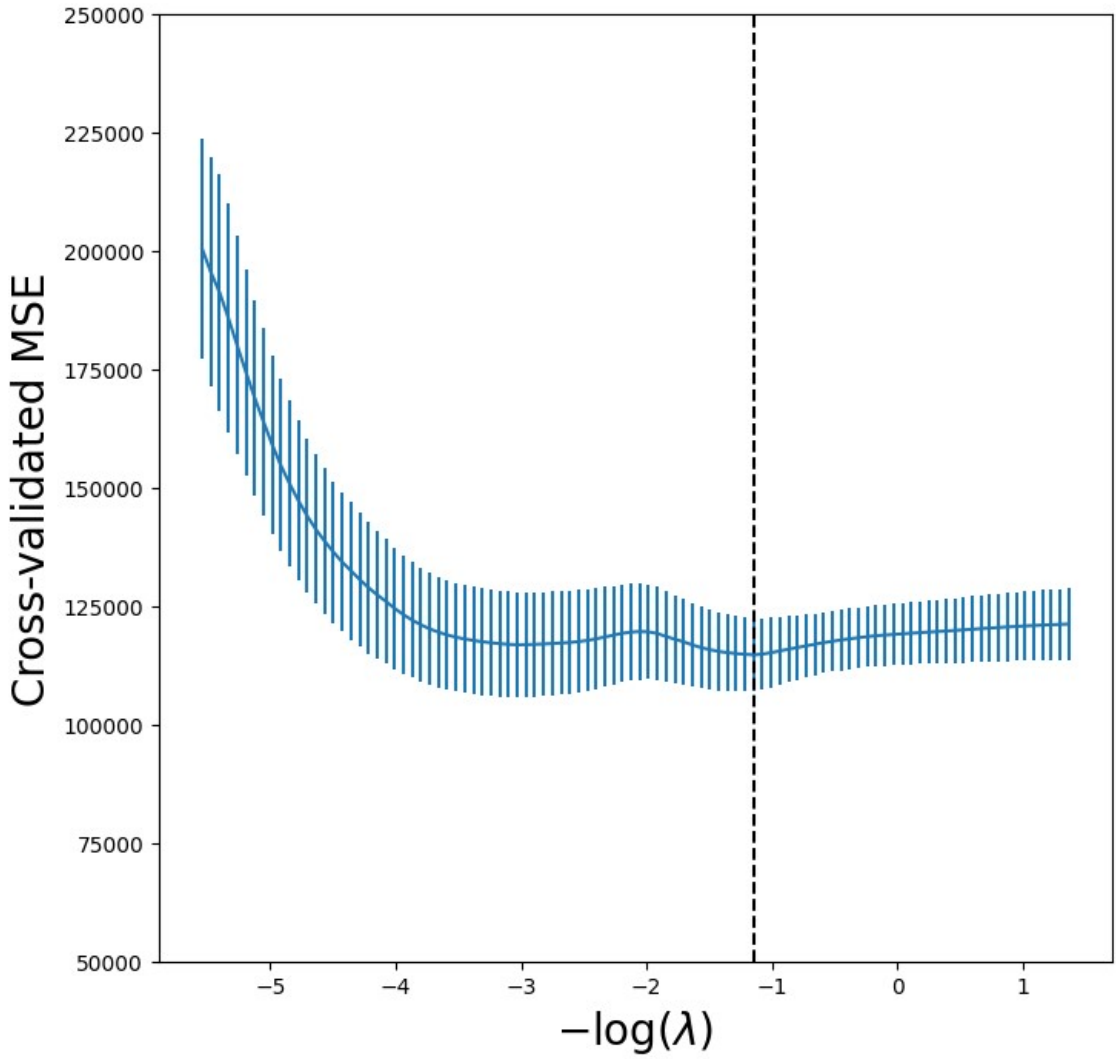


Figure 4: Cross-validation error against $\lambda$ in the log scale for the lasso

```
pca = PCA(n_components = 2)
linreg = skl.LinearRegression()
pipe = Pipeline([('scaler', scaler), ('pca', pca),
                 ('linreg', linreg)])
```

We use skm.GridSearchCV(sklearn.model_selection is abbreviated as skm) to perform cross-validation and choose number of principal components.

```
param_grid = {'pca__n_components': range(1, 20)}
grid = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=kfold,
                        scoring='neg_mean_squared_error')
grid.fit(X, Y)
```

Let us refer to the plot for cross-validation errors varying the number of components in figure 5.
We find minimum cross-validation error when number of components = 17. We also find from the chart,the cross-validation error is approximately same, if the number of components = 6 is included in the model.
A cumulative total of 88.63% of variance is explained for features and response by the first 6 principal

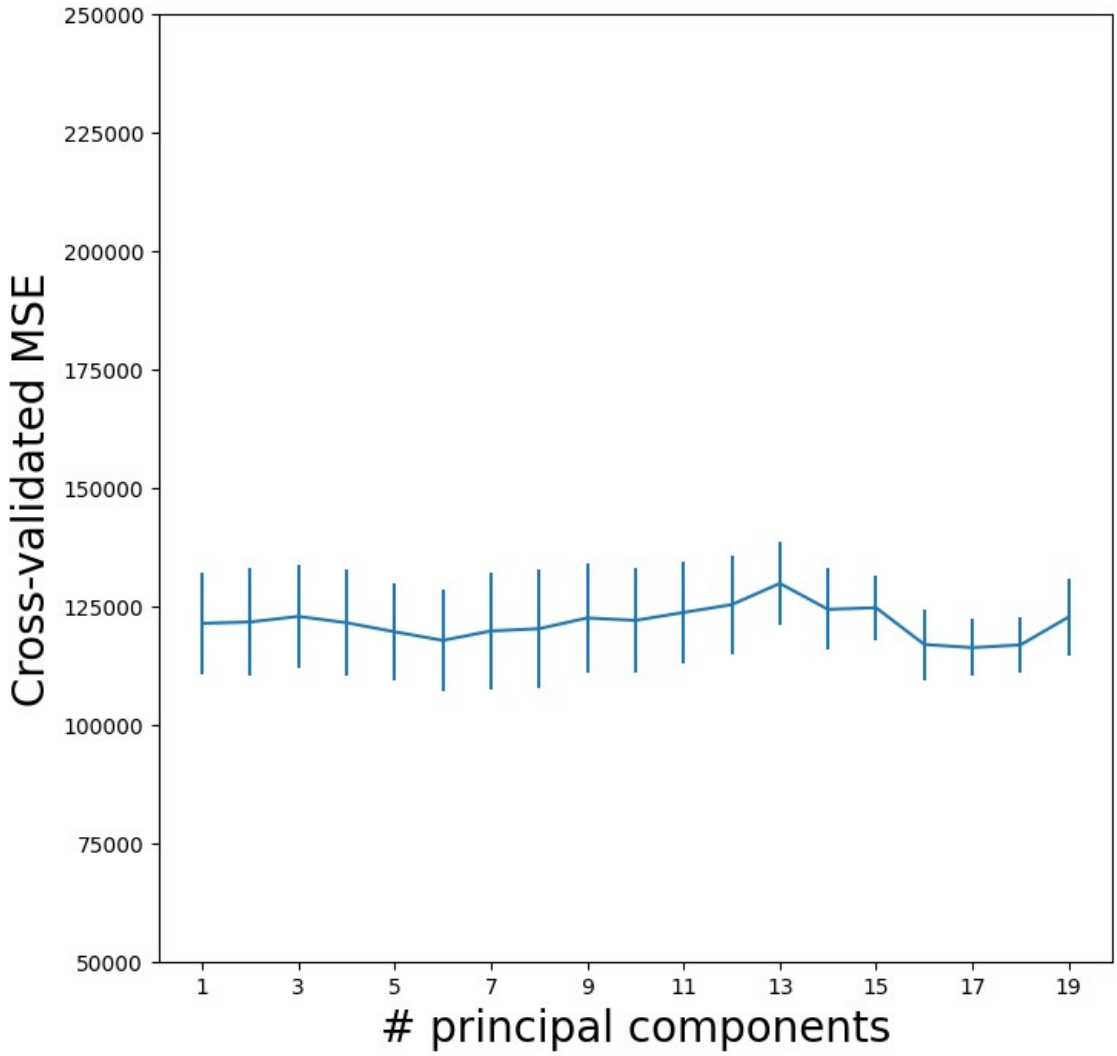


Figure 5: Cross-validation error against the number of components for PCA linear regression

components.

```
sum(pcaPipe.named_steps['pca'].explained_variance_ratio_)
0.8863477937701065
```

To evaluate the test error, we first split the data into test and training data set.
Then we fit the PCR with number of components = 6 on the training set.
The test MSE is 131688.545.

```
outer_valid = skm.ShuffleSplit(n_splits=1, test_size=0.25, random_state=1)
pcaCV = PCA(n_components = 6)
linreg = skl.LinearRegression()
pcaPipe = Pipeline([('scaler', scaler), ('pca', pcaCV),
                    ('linreg', linreg)])
pcaResults = skm.cross_validate(pcaPipe,
                                X,
                                Y,
                                cv = outer_valid,
                                scoring = 'neg_mean_squared_error')
-pcaResults['test_score']
```

```
array([131688.54491444])
```

The above test MSE is comparable to the results obtained using ridge regression and or the lasso. However, the PCR does not perform any kind of variable selection or directly produce coefficient estimates, so it's difficult to interpret the model.

### 1.1.4 Comparison of models deployed

Let us now review the results of the models deployed on the Hitters data set to predict salary of baseball players based on their performance statistics of the previous year. The table 1 summarizes the results for predicting salary of a baseball player.
The ridge regression test MSE of value 132393.84 is comparable to the lasso and PCR.The ridge regression does not perform variables selection, it shrinks all of the coefficients towards zero, but none of them is exactly zero.
The Lasso results the test MSE of value 133473.14, marginally higher than the ridge regression. However, the lasso provides sparse solution of coefficient estimates. The model includes non zero coefficient estimates of 13 out of 19 variables based on $\lambda$ chosen by cross-validation.
The PCR results the test MSE of value 131688.545 which has produced better result compared to both ridge regression and lasso on the Hitters data set. However, the final model produced by the PCR is more difficult to interpret as it does not perform selection among variables or directly produce coefficient estimates.

Table 1: Results for predicting salary of baseball player on the Hitters data set

| Methods | CV MSE | Test MSE | No. of predictors used |
|---|---|---|---|
| Ridge Regression ($L_2$ penalised) | 115526.71 | 132393.84 | 19 |
| Lasso ($L_1$ penalised) | 114690.73 | 133473.14 | 13 |
| Principal Components Regression (PCR) | 117752.65 | 131688.55 | 19 |

## 1.2 Classification

We will use the example based on real world data set, "Labeled Faces in the Wild", aka LFW from **?** to solve the face recognition problem of identifying an individual from the pixel of the image. We use the API, fetch_lfw_people from **?** to load the LFW people data set.

First we import the required libraries.

```
import matplotlib.pyplot as plt
import numpy as np

from sklearn.datasets import fetch_lfw_people
from sklearn.decomposition import PCA
import sklearn.linear_model as skl
import sklearn.model_selection as skm
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import ConfusionMatrixDisplay, classification_report
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
```

Let us extract the data set retaining pictures of people who have at least 70 different pictures by passing the argument, $min_faces_per_person = 70$ in the API, fetch_lfw_people. We have used ratio parameter, $resize = 0.4$ to resize the each face picture.

```
lfw_people = fetch_lfw_people(min_faces_per_person=70, resize=0.4)

# introspect the images arrays to find the shapes (for plotting
n_samples, h, w = lfw_people.images.shape

X = lfw_people.data
n_features = X.shape[1]

# the label to predict is the id of the person
y = lfw_people.target
target_names = lfw_people.target_names
n_classes = target_names.shape[0]

print("Total dataset size:")
print("n_samples: %d" % n_samples)
print("n_features: %d" % n_features)
print("n_classes: %d" % n_classes)

Total dataset size:
n_samples: 1288
n_features: 1850
n_classes: 7
```

We split the data into a training and a test set with 25% of the data for testing.

```
X_train, X_test, y_train, y_test =
    skm.train_test_split(X, y, test_size=0.25, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

### 1.2.1 Support Vector Classification (SVC) using Principal Component Analysis(PCA)

Let us compute principal components on the face data set for linear dimensionality reduction. To fit the support vector classifier, we have used SupportVectorClassifier() function (abbreviated as SVC()) from sklearn on transformed data of reduced dimension by principal components. We use Pipeline to chain PCA for data transformation and SVC for classification. We can refer API reference from **?** to learn various parameters of PCA and SVC APIs.

```
n_components = 2
pca =  PCA(n_components=n_components,svd_solver="randomized",whiten=True)
svc =  SVC(kernel = "rbf", class_weight="balanced", C=1.0)
pipe = Pipeline([('pca', pca),
                 ('classify', svc)])
```

We use skm.GridSearchCV to perform cross-validation for choosing the number of principal components. We set refit to a function named as best_low_complexity in GridSearchCV which returns the selected the index of a cross-validation for the best candidate parameter setting. The source code of the function,best_low_complexity is given in the appendix **??**.

```
param_grid = {'pca__n_components':range(1, 201,2)}

grid = skm.GridSearchCV(pipe,
                        cv=10,
                        n_jobs = 1,
                        param_grid = param_grid,
                        scoring="accuracy",
                        refit = best_low_complexity)
grid.fit(X_train, y_train)
```

Let us plot the cross-validation score for varying the number of principal components in figure 6 (source code in appendix **??**). According to the value of $best_index_ = 22$ from the cv_results_, the corresponding number of principal components = 45 and Cross-validation accuracy score = 84%.

```
best_index_ = grid.best_index_

print("The best_index_ is %d" % best_index_)
print("The n_components selected is %d" % n_components[best_index_])
print(
    "The corresponding accuracy score is %.2f"
    % grid.cv_results_["mean_test_score"][best_index_]
)

The best_index_ is 22
The n_components selected is 45
The corresponding accuracy score is 0.84
```

Let us evaluate the model performance on the test data set. We leverage sklearn.metrics library for quantitative evaluation. We use classification_report function to report classification metrics and ConfusionMa-

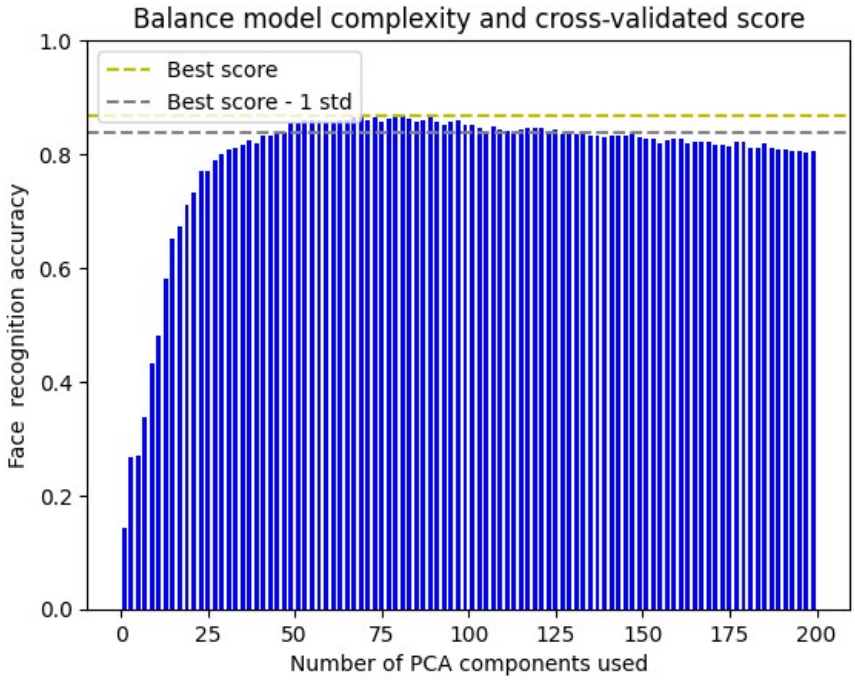


Figure 6: Cross-validation classification score for varying the number of principal components on LFW data set for face recognition

trixDisplay.from_estimator function to visualize confusion matrix shown in figure 7. The model produces overall classification accuracy 84% on the test data set.

```
print("Predicting people's names on the test set")
y_pred = grid.predict(X_test)

print(classification_report(y_test, y_pred, target_names=target_names))
ConfusionMatrixDisplay.from_estimator(
    grid, X_test, y_test, display_labels=target_names,
    xticks_rotation="vertical"
)
plt.tight_layout()
plt.show()
```

```
Predicting people's names on the test set
                   precision    recall  f1-score   support

     Ariel Sharon       0.82      0.69      0.75        13
     Colin Powell       0.84      0.93      0.88        60
  Donald Rumsfeld       0.67      0.59      0.63        27
    George W Bush       0.88      0.89      0.89       146
Gerhard Schroeder       0.69      0.80      0.74        25
      Hugo Chavez       0.91      0.67      0.77        15
       Tony Blair       0.88      0.81      0.84        36

         accuracy                           0.84       322
        macro avg       0.81      0.77      0.79       322
     weighted avg       0.84      0.84      0.84       322
```

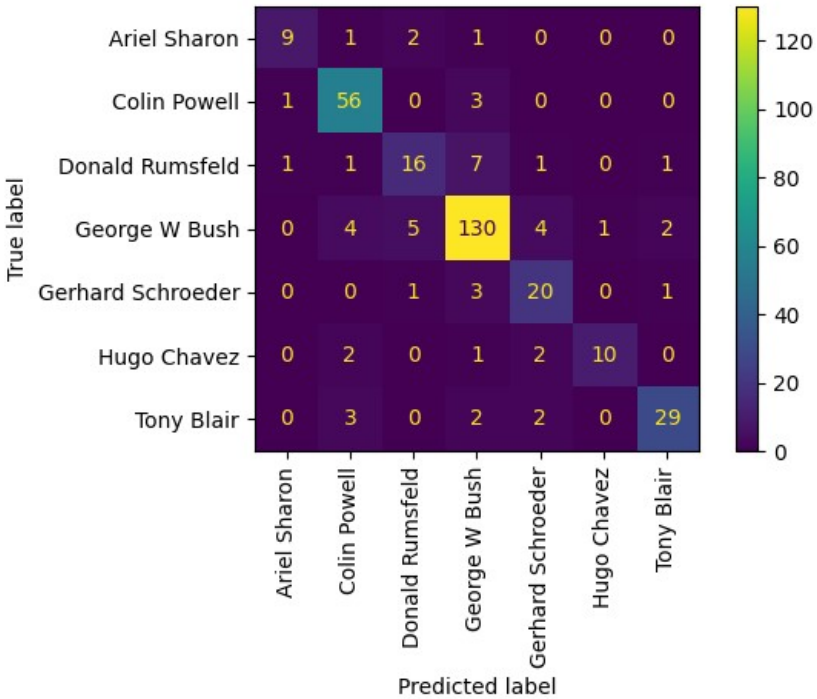


Figure 7: Confusion Matrix for face recognition using multi lebel Support Vector Classification on the principal components of the LFW data set

### 1.2.2 Logistic Regression - $L_1$ Regularization

We will fit logistic regression with a $L_1$ regularization (penalty) term on the LFW data set to identify an individual based on face recognition.
We use the LogisticRegressionCV function from sklearn.linear_model library for model fit using cross validation. We apply cross-validation to choose a value of C (i.e., inverse of regularization parameter) that results the best classification score.

```
clf = skl.LogisticRegressionCV(penalty='l1',solver='saga',
        scoring='accuracy', tol=1e-3, Cs = 20, class_weight='balanced',
        cv=10, random_state=42, max_iter=300)
clf.fit(X_train, y_train)
```

The best cross-validation score is 82.4% for $C = 5$.
We get the best score by running np.max(clf.scores_[0].mean(0)) and $C$ by running clf.C_.

```
print("The best accuracy score %f" % np.max(clf.scores_[0].mean(0)))
print("C (i.e., inverse of regularization parameter)
       selected is %f" % clf.C_[0])

The best accuracy score 0.823969
C (i.e., inverse of regularization parameter) selected is 4.281332
```

We use classification_report function to report classification metrics. The model produces overall classification accuracy 84% on the test data set. The confusion matrix is shown in figure 8

```python
print("Predicting people's names on the test set using" + '\n'+
"Logistic Regression - L1 Regularization")
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred, target_names=target_names))
ConfusionMatrixDisplay.from_estimator(
    clf, X_test, y_test, display_labels=target_names,
    xticks_rotation="vertical"
)
plt.tight_layout()
plt.show()
```

```
Predicting people's names on the test set using
Logistic Regression - L1 Regularization
                   precision    recall  f1-score   support

     Ariel Sharon       0.59      0.77      0.67        13
     Colin Powell       0.83      0.90      0.86        60
  Donald Rumsfeld       0.65      0.74      0.69        27
    George W Bush       0.97      0.85      0.91       146
Gerhard Schroeder       0.73      0.96      0.83        25
      Hugo Chavez       0.71      0.67      0.69        15
       Tony Blair       0.79      0.75      0.77        36

         accuracy                           0.84       322
        macro avg       0.75      0.81      0.77       322
     weighted avg       0.85      0.84      0.84       322
```

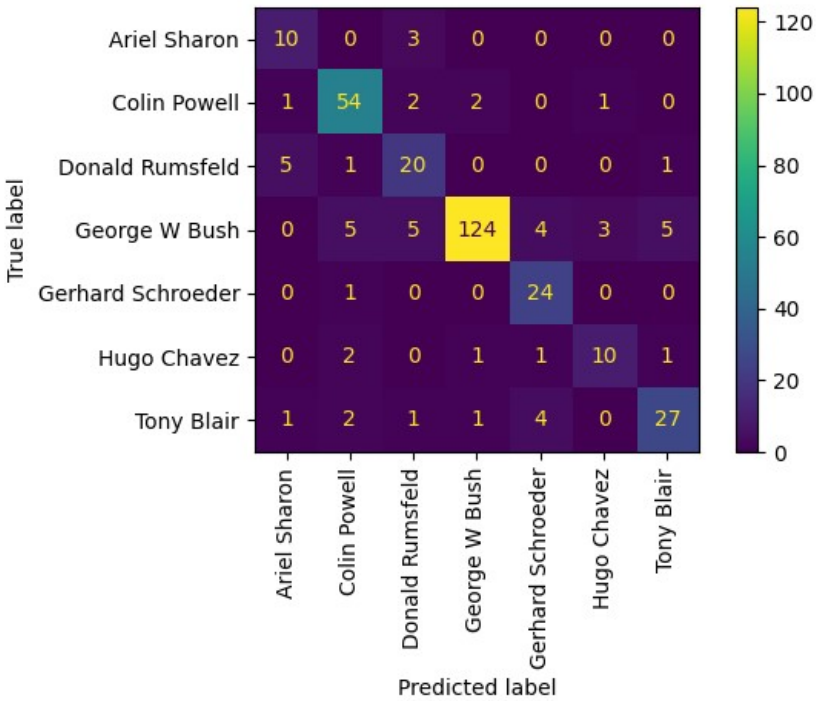


Figure 8: Confusion Matrix for face recognition on the LFW data set using Logistic Regression - $L_1$ Regularization

### 1.2.3 Logistic Regression - $L_2$ Regularization

We will fit logistic regression with $L_2$ regularization on the LFW data set to identify an individual by recognizing the face of the individual.
We use $penalty =' l2'$ in LogisticRegressionCV function. We choose $C$ using cross-validation.

```
clf_ridge = skl.LogisticRegressionCV(penalty='l2',solver='saga', tol=1e-3,
        Cs = 20, class_weight='balanced',cv=10,
        random_state=42, max_iter=300)
clf_ridge.fit(X_train, y_train)
```

The best cross-validation score is 84% for $C = 0.09$.
We get the best score by running np.max(clf_ridge.scores_[0].mean(0)) and $C$ by running clf_ridge.C_.

```
print("The best accuracy score %f" % np.max(clf_ridge.scores_[0].mean(0)))
print("C (i.e., inverse of regularization parameter)
selected is %f" % clf_ridge.C_[0])

The best accuracy score 0.837457
C (i.e., inverse of regularization parameter) selected is 0.088587
```

We use classification_report function to report classification metrics. The model produces overall classification prediction accuracy 82% on the test data set. The confusion matrix is shown in figure 9.

```
print("Predicting people's names on the test set using" + '\n'+
"Logistic Regression - $L_2$ Regularization")
y_pred = clf_ridge.predict(X_test)

print(classification_report(y_test, y_pred, target_names=target_names))
ConfusionMatrixDisplay.from_estimator(
    clf_ridge, X_test, y_test, display_labels=target_names, xticks_rotation="
)
plt.tight_layout()
plt.show()

Predicting people's names on the test set using
Logistic Regression - L2 Regularization
                   precision    recall  f1-score   support

     Ariel Sharon       0.64      0.69      0.67        13
     Colin Powell       0.80      0.88      0.84        60
  Donald Rumsfeld       0.61      0.70      0.66        27
    George W Bush       0.95      0.84      0.89       146
Gerhard Schroeder       0.69      0.88      0.77        25
      Hugo Chavez       0.65      0.73      0.69        15
       Tony Blair       0.79      0.75      0.77        36

         accuracy                           0.82       322
        macro avg       0.73      0.78      0.75       322
     weighted avg       0.83      0.82      0.82       322
```

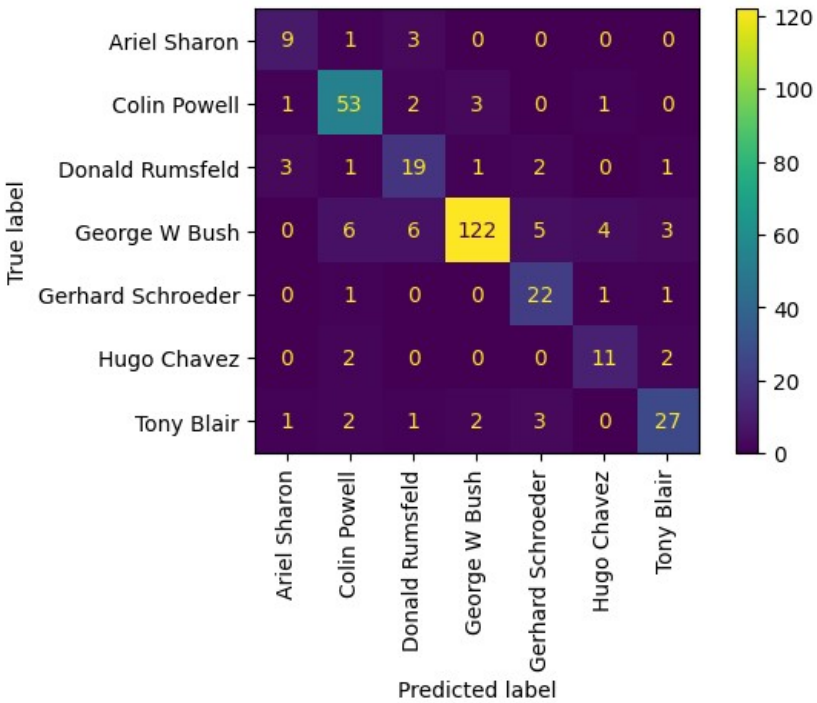


Figure 9: Confusion Matrix for face recognition on the LFW data set using Logistic Regression - L2 Regularization

### 1.2.4 Comparison of models deployed

Let us now review the results of the models deployed on the LFW data set to identify an individual by recognizing the face of the individual. The table 2 summarizes the results of the accuracy of the prediction of multi-label classification.

We find that the CV accuracy of SVC based on PCA is higher than the logistic regression with $L_1$ regularization, however, the test accuracy is the same for both the models. The CV accuracy of the logistic regression with $L_2$ regularization matches with SVC based on PCA, but it produces lower test accuracy compared to both SVC based on PCA and logistic regression with $L_1$ regularization.

Clearly, the logistic regression with $L_1$ (lasso) regularization allows more variation outside the training data compared to $L_2$ regularization even though the former is more biased compared to other two models deployed. The logistic regression with $L_1$ regularization (lasso) also provides an advantage of sparse coefficient estimates.

Table 2: Results of classification prediction accuracy on the LFW data set

| Methods | CV Classification Accuracy | Test Classification Accuracy | No.of features used |
|---|---|---|---|
| PCA based Support Vector Classification | 84% | 84% | No. of PCs= 45 on 1850 features |
| Logistic Regression with Lasso $L_1$ Regularization | 82.4% | 84% | Coefficient of features for 7 classes:80-426 out of 1850 coefficients set to zero across 7 classes |
| Logistic Regression with Lasso $L_2$ Regularization | 84% | 82% | Full set of 1850 features |